

Turing Machine Simulator and Busy Beaver Project

Project Documentation

1. Introduction

The Turing Machine is one of the most fundamental models of computation in theoretical computer science. Introduced by Alan Turing in 1936, it provides a simple yet powerful abstract model capable of representing any algorithmic computation. Despite its minimal structure—consisting of a tape, a head, and a set of transition rules—it forms the theoretical foundation of modern computers.

The goal of this project is to implement a complete **Turing Machine simulator in Python** and use it to demonstrate computational processes. The simulator is then applied to perform mathematical operations such as **unary addition** and **unary multiplication**, and finally to explore the well-known theoretical problem called the **Busy Beaver problem**.

The project emphasizes understanding computation through state transitions, tape manipulation, and algorithmic simulation.

2. Project Structure

The project consists of several Python files, each responsible for a specific component of the simulation.

turing_machine.py

This file contains the core implementation of the Turing Machine simulator.

It defines the `TuringMachine` class and provides the functionality for executing and debugging Turing machine programs.

test_turing_adder.py

This script defines a Turing machine that performs **addition using unary representation**. It provides several test inputs to verify that the simulator works correctly.

test_turing_multiplier.py

This file implements a more complex Turing machine that performs **multiplication using unary numbers**. It uses several auxiliary tape symbols to manage intermediate steps during the multiplication process.

busy_beaver.py

This file demonstrates the **Busy Beaver problem** by running several Busy Beaver machines with different numbers of states and reporting the number of steps and symbols written before halting.

3. Turing Machine Simulator Design

3.1 Class Structure

The simulator is implemented using the `TuringMachine` class. The class receives a **transition dictionary** describing the behavior of the machine.

Each transition has the form:

$(\text{state}, \text{symbol}) \rightarrow (\text{next_state}, \text{write_symbol}, \text{direction})$

Where:

- **state** is the current state of the machine.
- **symbol** is the symbol currently under the head.
- **next_state** is the state after the transition.
- **write_symbol** is the symbol written to the tape.
- **direction** determines the head movement (Left or Right).

3.2 Tape Representation

The tape is modeled using three components:

- **left_hand_side** – list of symbols to the left of the head (stored in reverse order)
- **symbol** – the current symbol under the head
- **right_hand_side** – list of symbols to the right of the head

This structure allows efficient simulation of head movement and tape extension.

3.3 Generator-Based Execution

The `run()` method is implemented as a **Python generator**. Instead of computing the entire execution at once, it yields the configuration of the machine after every step.

Each yielded configuration contains:

- current state
- left side of the tape
- current symbol under the head
- right side of the tape

This design allows step-by-step monitoring and debugging of the computation.

3.4 Acceptance and Rejection

Two helper methods are provided:

accepts(input, step_limit)

Runs the machine for a maximum number of steps and returns:

- True if the machine reaches the accept state
- False if the machine rejects
- None if the step limit is reached without halting

rejects(input)

Returns the opposite result of accepts.

3.5 Debug Mode

The debug() function prints the configuration of the machine at each step in the format:

state left[symbol]right

This visualization is extremely useful for understanding how the tape evolves during computation.

4. Example Turing Machines

4.1 test_turing_machine_example1

This machine is extremely small and acts as a strict recognizer for one very specific input pattern. Its transition table is:

- ("q0", "#") -> ("saw_#", "#", "R")
- ("saw_#", "#") -> ("saw_##", "#", "R")
- ("saw_##", "") -> ("qa", "", "R")

What it does

It accepts only the input ##.

How it works

1. In state q0, it expects to read #.
2. It moves right and enters saw_#.
3. In saw_#, it expects another #.
4. It moves right and enters saw_##.
5. If the next symbol is blank, it accepts.

Computation meaning

This is not an arithmetic machine or a general parser. It is a very small language recognizer for the single string ##. Any extra symbols, missing symbols, or different characters cause rejection.

Interpretation

You can describe it as:

- a two-symbol delimiter checker
- a machine that accepts exactly one pattern
- a trivial example showing how state transitions and acceptance work

4.2 test_turing_machine_example2

This machine is more interesting. It uses markers like X to compare symbols on both sides of a delimiter #.

What it does

It checks whether the symbols on the left and right side of # match in the intended paired structure. In the examples, it behaves like a **matching / equality-checking machine around a separator**.

- ("q0", "#") -> ("End", "#", "R")
- ("End", "") -> ("qa", "", "R")
- ("q0", "0") -> ("FindDelimiter0", "X", "R")
- ("FindDelimiter0", "#") -> ("Check0", "#", "R")
- ("Check0", "0") -> ("FindLeftmost", "X", "L")
- ("q0", "1") -> ("FindDelimiter1", "X", "R")
- ("FindDelimiter1", "#") -> ("Check1", "#", "R")
- ("Check1", "1") -> ("FindLeftmost", "X", "L")
- ("FindLeftmost", "0") -> ("FindLeftmost", "0", "L")
- ("FindLeftmost", "1") -> ("FindLeftmost", "1", "L")
- ("FindLeftmost", "X") -> ("FindLeftmost", "X", "L")
- ("FindLeftmost", "#") -> ("FindLeftmost", "#", "L")
- ("FindLeftmost", "") -> ("FindNext", "", "R")
- ("FindNext", "X") -> ("FindNext", "X", "R")
- ("FindNext", "0") -> ("FindDelimiter0", "X", "R")
- ("FindNext", "1") -> ("FindDelimiter1", "X", "R")
- ("FindNext", "#") -> ("End", "#", "R")
- ("FindDelimiter0", "0") -> ("FindDelimiter0", "0", "R")
- ("FindDelimiter0", "1") -> ("FindDelimiter0", "1", "R")
- ("FindDelimiter1", "0") -> ("FindDelimiter1", "0", "R")
- ("FindDelimiter1", "1") -> ("FindDelimiter1", "1", "R")
- ("Check0", "X") -> ("Check0", "X", "R")
- ("Check1", "X") -> ("Check1", "X", "R")
- ("End", "X") -> ("End", "X", "R")

Core idea

1. Start from the left side.
2. Find a symbol 0 or 1.
3. Mark it as X.
4. Move across # to the corresponding side.
5. Check whether the matching symbol is the same.

6. Mark matched symbols and repeat.
7. Accept when everything has been matched properly.

Important states

- q0: starting state
- FindDelimiter0, FindDelimiter1: move right looking for the delimiter and matching symbol
- Check0, Check1: verify the corresponding symbol
- FindLeftmost: move back to the leftmost part of the tape
- FindNext: locate the next unprocessed symbol
- End: finish and accept

How it works

- **From the left side**, the machine picks the first unmarked symbol (0 or 1) and replaces it with X.
- It then **moves right to the #**, crosses it, and searches for the **first unmatched symbol** on the right side.
- It checks whether it **matches** the symbol chosen on the left:
- If it matches → the machine marks it with X.
- If it does NOT match → the machine rejects.
- After marking the matching symbol on the right, it **returns to the left end** of the tape.
- The machine repeats:
 - Find next unmarked left-side symbol.
 - Match it with the corresponding right-side symbol.
- When the machine reaches # on the left side (meaning all are marked), it moves to the right to verify that the entire right side is also fully marked.
- If everything is marked correctly, the machine **accepts**.

Computation meaning

This machine is doing a form of **symbol-by-symbol consistency checking** between the parts of the tape separated by #. It is essentially a structured matcher using X as a visited marker.

5. Unary Arithmetic with Turing Machines

5.1 Unary Representation

In unary representation, numbers are represented by repeating the symbol **1**.

Examples:

$0 \rightarrow (\text{empty})$

$1 \rightarrow 1$

$2 \rightarrow 11$

$3 \rightarrow 111$

$4 \rightarrow 1111$

When two numbers are given to the machine, they are separated by the symbol **0**.

Example:

$2 + 3 \rightarrow 110111$

6. Unary Addition Machine

The addition machine is implemented in `test_turing_adder.py`.

Algorithm Idea

The machine performs the following operations:

1. Move right across the first sequence of 1s.
2. Replace the separator 0 with 1.
3. Continue moving right until the end of the tape.
4. Remove the final 1 to normalize the result.
5. Enter the accept state.

This effectively merges the two unary numbers.

Example Computations

Input:

110111

Meaning:

$2 + 3$

Output tape:

11111

Meaning:

5

Input:

11101111

Meaning:

3 + 4

Output:

1111111

Meaning:

7

Input:

0111

Meaning:

0 + 3

Output:

111

Meaning:

3

7. Unary Multiplication Machine

The multiplication machine in `test_turing_multiplier.py` is significantly more complex.

Algorithm Idea

The machine multiplies two unary numbers by **repeated copying**.

It uses several auxiliary symbols:

- **X** – marks processed symbols
- **Y** – temporary marker during copying
- **Z** – intermediate marker

Steps of the algorithm:

1. Select a 1 from the first number.
2. Traverse to the second number.
3. Copy the second number to the end of the tape.
4. Repeat this process for each 1 in the first number.
5. Clean up temporary markers.
6. Halt in the accept state.

Example Computations

Input:

110111

Meaning:

2×3

Output:

111111

Meaning:

6

Input:

11101111

Meaning:

3×4

Output:

111111111111

Meaning:

12

Input:

101111

Meaning:

1×4

Output:

1111

Meaning:

4

8. Busy Beaver Problem

The **Busy Beaver problem** is a well-known problem in computability theory.

Given a Turing machine with a fixed number of states and a binary alphabet, the goal is to find the machine that writes the **maximum number of 1s on the tape before halting**.

This value is denoted as:

$BB(n)$

where **n** is the number of states.

Why the Busy Beaver Problem is Difficult

The Busy Beaver problem is **undecidable**. Determining the optimal machine for arbitrary numbers of states would require solving the **Halting Problem**, which has been proven impossible for general cases.

As the number of states increases, the number of possible machines grows extremely rapidly.

Busy Beaver Implementation

The file `busy_beaver.py` contains implementations of several Busy Beaver machines with different numbers of states.

Each program is defined using compact transition rules such as:

`a0 → b1r`

meaning:

state a reading 0 → write 1, move right, go to state b.

Example Results

Typical known results are approximately:

States | Maximum 1s Written | Steps Before Halting

2 | 4 | 6

3 | 6 | 13

4 | 13 | 107

5 | 4098 | 47,176,870

These results illustrate how rapidly the complexity grows.

9. Advantages of Using Python Generators

Using **Python generators** in the simulator provides several benefits:

1. **Memory efficiency** – configurations are generated one step at a time.
2. **Step-by-step debugging** – intermediate states can easily be inspected.
3. **Controlled execution** – simulations can be stopped early if necessary.
4. **Visualization support** – configurations can be printed during execution.

This approach is particularly important for simulations that may run for millions of steps, such as Busy Beaver machines.

10. Busy Beaver Results

10.1 2-state Busy Beaver simulation

The 2-state Busy Beaver my Turing machine simulator using the transition table in `busy_beaver.py`:

```
{  
    "a0": "b1r",  
    "b0": "a1l",  
    "a1": "b1l",  
    "b1": "h1r",  
}
```

Halted after **6 steps** and wrote **4 ones** on the tape.

10.2 Comparison with the known 2-state Busy Beaver

This result matches the classical optimal 2-state Busy Beaver value:

- **Steps:** 6
- **Ones written:** 4

Therefore, the 2-state program is **optimal** and agrees exactly with the best known 2-state Busy Beaver machine.

10.3 3-state Busy Beaver

Transition table:

```
{  
    "a0": "b1r",  
    "a1": "c1l",  
    "b1": "b1r",  
    "b0": "a1l",  
    "c0": "b1l",  
    "c1": "h1r",  
}
```

Observed output:

- **Steps:** 13
- **Ones written:** 6

10.4 4-state Busy Beaver

Transition table:

```
{
    "a0": "b1r",
    "a1": "b1l",
    "b0": "a1l",
    "b1": "c0l",
    "c0": "h1r",
    "c1": "d1l",
    "d0": "d1r",
    "d1": "a0r",
}
```

Observed output:

- **Steps:** 107
- **Ones written:** 13

These are the maximum outputs produced by the programs for 3 and 4 states.

10.5 5-state Busy Beaver program

5-state Busy Beaver program already present in `busy_beaver.py`

```
{
    "a0": "b1r",
    "a1": "c1l",
    "b0": "c1r",
    "b1": "b1r",
    "c0": "d1r",
    "c1": "e0l",
    "d0": "a1l",
    "d1": "d1l",
    "e0": "h1r",
    "e1": "a0l",
}
```

The simulation showed that this machine halted after **47,176,870 steps** and wrote **4,098 ones**.

This is a valid halting Busy Beaver candidate, and it is the well-known classic 5-state champion machine. Based on the observed halting behavior, the machine is not infinite-looping; it halts after a very large number of steps.

10.6 Comparison with Busy Beaver Wiki

Compared with the standard known best results from Busy Beaver research, the outputs are:

- **2 states:** 6 steps, 4 ones
- **3 states:** 13 steps, 6 ones
- **4 states:** 107 steps, 13 ones
- **5 states:** 47,176,870 steps, 4,098 ones

Evaluation

- For **2 states**, my result is exactly optimal.
- For **3 states**, the program matches the known maximum number of ones written for the classic 3-state setting used in the assignment context.
- For **4 states**, the result matches the classical best-known machine.
- For **5 states**, the result matches the classic 5-state Busy Beaver champion machine.

In conclusion, the machines implemented did reproduce the standard Busy Beaver results for 2, 3, 4, and 5 states, and the simulator behaves correctly on these benchmark programs.

11. Conclusion

This project demonstrates the power and flexibility of Turing machines as a model of computation. By implementing a Turing machine simulator in Python, we were able to explore fundamental computational concepts and implement arithmetic operations using simple state transitions. The project also highlights the difference between **simple machine** definitions and **complex computational behavior**, especially when examining problems such as the Busy Beaver.

The empirical analysis of the 1 to 5-state Busy Beaver (BB) problem illustrates the limits of computability and the sheer scale of the Halting Problem. While the 2-state machine halts in only 6 steps writing 4 ones, increasing the state count to 5 yields a massive explosion in complexity, requiring over 47 million steps (47,176,870) to write 4,098 ones before halting. This extreme, non-computable rate of growth demonstrates how quickly Turing machines transition from trivial configurations to highly complex, long-running dynamical systems.

Through this implementation, we gain deeper insight into how computation can emerge from simple rules, how resource-efficient simulation techniques (such as Python generators) enable the execution of millions of steps, and how theoretical limitations, such as undecidability, arise in computer science.